# Programowanie Aplikacji INternetowych

# strona vs aplikacja internetowa

strona internetowa to dokument, a aplikacja internetowa to program dostępny w przeglądarce

# przegladarka

- program do przeglądania stron i aplikacji internetowych
- komunikuje sie z serwerem - pobiera i wysyła zasoby
- interpretuje HTML, CSS i JavaScript
- umożliwia wykonywanie skryptów
- przykłady: Chrome, Firefox, Edge, Safari
- DevTools - narzędzia dla programisty

# skrypt

- program wykonywany w określonym środowisku
- zwykle nie wymaga klasycznego procesu kompilacji przed uruchomieniem
- może działać po stronie klienta lub serwera
- skrypt może m.in. przetwarzać dane, reagować na działania użytkownika, komunikować się z serwerem

# środowisko uruchomieniowe

- środowisko umożliwiające wykonywanie programu/skryptu
- dostarcza potrzebnych mechanizmów i bibliotek
- określa, w jakich warunkach działa kod
- zapewnia dostęp do wybranych zasobów systemu

# napisz swój pierwszy skrypt

Utwórz plik index.html z podstawową strukturą strony

1. <script> console.log("Witaj szkoło!"); </script>
2. <script src="skrypt.js"></script>

# co się dzieje w przeglądarce?

# dynamiczne typowanie

```
let x = “10”
console.log(x+5);

console.log(x == 10);
console.log(x === 10);
```

# let, const, var

```
if (true) {
    let x = 10;
}

console.log(x);
```

```
if (true) {
    var x = 10;
}

console.log(x);
```

# typy danych

```
let age = 18;
let price = 19.99;
let name = "Milena";
let isStudent = false;
let value = null;
let x;

console.log(typeof age);
```

# konwersja typów - cast

```
let text = "123";
let number = Number(text);
let text2 = number.toString();

console.log(number);
console.log(typeof number);
```

# funkcje

```
function hello(name) {
    console.log("Hello " + name);
}

const function = function(name) { console.log("Hello " +
name); };

hello("Milena");
```

# funkcje strzałkowe

```
const dodawanie = (a, b) => {
    return a + b;
};

const dodawanie = (a, b) => a + b;
```

# zadanie!

Zadeklaruj zmienne:
let username = "Kuba";
let age = "17";
let password = "abc12345";

Napisz funkcję isValidUser(username, age, password)która zwraca true jeżeli:

- nazwa użytkownika ma co najmniej 3 znaki,
- użytkownik ma co najmniej 13 lat,
- hasło ma co najmniej 8 znaków.

W przeciwnym wypadku zwraca false. Przetestuj funkcję dla zadeklarowanych zmiennych.

# zadanie!

Napisz funkcję w języku JavaScript, która dla zadeklarowanych zmiennych stałych **month** i **year** zwróci liczbę dni danego miesiąca. Jeśli podanym miesiącem jest luty, funkcja musi sprawdzić, czy podany rok jest przestępny. Rok jest przestępny, jeśli jest podzielny przez 4 i nie jest podzielny przez 100, LUB jest podzielny przez 400. Zwrócony wynik działania funkcji wypisz w konsoli w postaci zdania np. “Luty 2024 ma 29 dni.”

# zadanie!

Dana jest tablica, w której każdy element to podany wiek osoby z grupy, która kupuje bilety do ZOO:

const **daneWejsciowe** = [2, "12", "30", "67", "tekst", null, 18];

Napisz funkcję, która dla każdego elementu tablicy obliczy cenę biletu na podstawie poniższych reguł.

- >3 lat wejście darmowe
- 3-18 lat bilet ulgowy 15 zł
- 65+ lat bilet seniora 20 zł
- bilet normalny 30 zł

# operacje na tablicach - map

Tworzy nową tablicę poprzez wykonanie funkcji na każdym elemencie tablicy. Argumentem jest funkcja (nie wywołanie funkcji!)

```
const numbers = [4, 9, 16, 25];
const newArr = numbers.map(Math.sqrt)
```

# operacje na tablicach - filter

Tworzy nową tablicę ze wszystkimi elementami tablicy wejściowej, które przejdą test.

```
const ages = [32, 33, 16, 40];
const result = ages.filter(checkAdult);

function checkAdult(age) {
  return age >= 18;
}
```

# operacje na tablicach - find

Zwraca wartość pierwszego elementu, który przejdzie test.
W przypadku nie znalezienia zwraca undefined.

```
const ages = [3, 10, 18, 20];

function checkAge(age) {
  return age > 18;
}

ages.find(checkAge);
```

# zadanie!

```
const prices = [12, 45, 8, 120, 35, 200, 17, 60];
```

1. Utwórz nową tablicę zawierającą ceny wszystkich produktów powiększone o 23% podatku VAT.
2. Utwórz nową tablicę zawierającą wyłącznie ceny produktów droższych niż 50 zł.
3. Znajdź pierwszą cenę w tablicy, która jest większa niż 100 zł
4. Znajdź pierwszą cenę w tablicy, która jest większa niż 1000 zł. Jeśli program nie znajdzie odpowiedniej ceny, wyświetl komunikat „Nie znaleziono produktu droższego niż 1000 zł”.

# obiekty w javascript

Obiekt to zbiór powiązanych ze sobą danych:

```
const uczen = {
    name: "Kasia",
    class: "4b",
    grades: [5, 4, 3, 5]
};
```

# properties

odczytywanie wartości:
console.log(student.name);
console.log(student[“class”]);

odczytywanie wartości poprzez zmienną:
const propertyName = “grades”;
console.log(student[propertyName]);

modyfikacja wartości:
student.name = “Basia”;
student.school = “SCI”;

# metody obiektów

```
const product = {
    name: "Monitor",
    price: 900,
    applyDiscount(percent) {
        this.price = this.price * (1 - percent / 100);
    }
};

console.log(product.price);
product.applyDiscount(10);
console.log(product.price);
```

# zagnieżdżone obiekty

```
const order = {
    number: 2137,
    customer: {
        name: "Milena",
        city: "Szczecin"
    },
    products: [
        { name: "Mysz", price: 80, quantity: 2 },
        { name: "Monitor", price: 900, quantity: 1 }
    ]
};

console.log(order.customer.name);
console.log(order.products[0].price);
```

# zadanie!

Stwórz tablicę **products** będącą zbiorem produktów pewnego sklepu internetowego. Każdy produkt ma swoje id, nazwę, cenę, ilość danych produktów w magazynie i minimalną ilość. Dodaj minimum 3 różne produkty do tablicy.

1. Oblicz i zwróć wartość wszystkich produktów z tablicy (suma cena * ilość).
2. Stwórz wyświetlony w konsoli raport o niskich stanach magazynowych, który wyświetli wszystkie produkty których ilość w magazynie jest mniejsza niż minimalna ilość. Zadbaj o odpowiednia informację dla użytkownika.

# porównywanie obiektów

```
const a = { price: 100 };
const b = { price: 100 };
const c = a;

console.log(a === b); // false
console.log(a === c); // true
```

# kopiowanie obiektów

```
const original = {
    name: "Laptop",
    price: 3000
};

const copy = { ...original };

copy.price = 2500;

console.log(original.price); // 3000
console.log(copy.price);     // 2500
```

# płytka vs głęboka kopia

Spread (...) tworzy płytką (shallow) kopię obiektu. Jeśli obiekt zawiera zagnieżdżone obiekty lub tablice, w kopii przechowywane są jedynie referencje do nich (czyli wskazują na te same miejsca w pamięci co oryginał).

Głęboką (deep) kopię można uzyskać np. poprzez zamianę na obiekt JSON:
let cloneUser = JSON.parse(JSON.stringify(userDetails));

# przekazywanie obiektów

Obiekt przekazany do funkcji może zostać zmodyfikowany przez tę funkcję.

```
const product = { name: "Laptop", price: 3000};

function calculateVATPrice(product) {
    return product.price * (1 + 23 / 100);
}
```

# zadanie!

Napisz funkcje lub metody do operacji na tablicy **products** z poprzedniego zadania.

1. findProductById - zwraca produkt o podanym identyfikatorze (jeśli nie znajdzie - undefined).
2. searchProducts(string) - wyszukuje produkty, których nazwa zawiera podany fragment tekstu, niezależnie od wielkości liter np. argument “mysz” zwraca produkt “Mysz gamingowa”.
3. addProduct(id, name, price, quantity, minimum) - która dodaje nowy produkt do tablicy (*array push*)

# try, catch

Jeśli chcemy przeprowadzić operację, która może się nie udać, możemy użyć try.

```
try {
    console.log("Wykonuję operację");
    throw new Error("Awaria");
} catch (error) {
    console.log(error.message);
}
```

# finally

Blok finally wykonuje się po try i catch niezaleznie czy zostały wyłapane jakieś błędy.

```
try {
    console.log("Wykonuję operację");
    throw new Error("Awaria");
} catch (error) {
    console.log(error.message);
} finally {
    console.log("Koniec operacji");
}
```

# zadanie!

Dodaj do funkcji x wyłapywanie błędnie podanych danych wejściowych, np. x(10, 2, 44). Po prostu wywal funkcję i przechwyć błąd.

# funkcja generator, yield

```
function* gen(index) {
  while (index < 2) {
    yield index;
    index++;
  }
}

const iterator = gen(0);

console.log(iterator.next().value);
console.log(iterator.next().value);
```

# definiowanie własnych obiektów

Deklaracja klasy:

```
class Rectangle {
    constructor(height, width) {
        this.height = height;
        this.width = width;
    }
}
```

```
class Rectangle {
    constructor(height, width) {
        this.height = height;
        this.width = width;
    }
    get area() {
        return this.calcArea();
    }
    calcArea() {
        return this.height * this.width;
    }
    *getSides() {
        yield this.height;
        yield this.width;
        yield this.height;
        yield this.width;}
}
```

getter

metoda

generator

```
const square = new Rectangle(10, 10);

console.log(square.area);
console.log([...square.getSides()]);
```

# zadanie!

Zdefiniuj klasę **Task**, która reprezentuje pojedyncze zadanie do wykonania i posiada **tytuł, priorytet**(1-3) i flagę **isCompleted**. Konstruktor przyjmuje nazwę i priorytet. Początkowo zadanie nie jest wykonane. Stwórz następujące metody:

- **complete**() - oznacza zadanie jako wykonane
- **changePriority**(newPriority) - zmienia priorytet na podany
- **getInfo**() - zwraca tekst zawierający nazwę, priorytet oraz status zadania

Test nowej klasy:

```
const task1 = new Task("Przygotować projekt", 3);
const task2 = new Task("Nauczyć się JavaScriptu", 2);

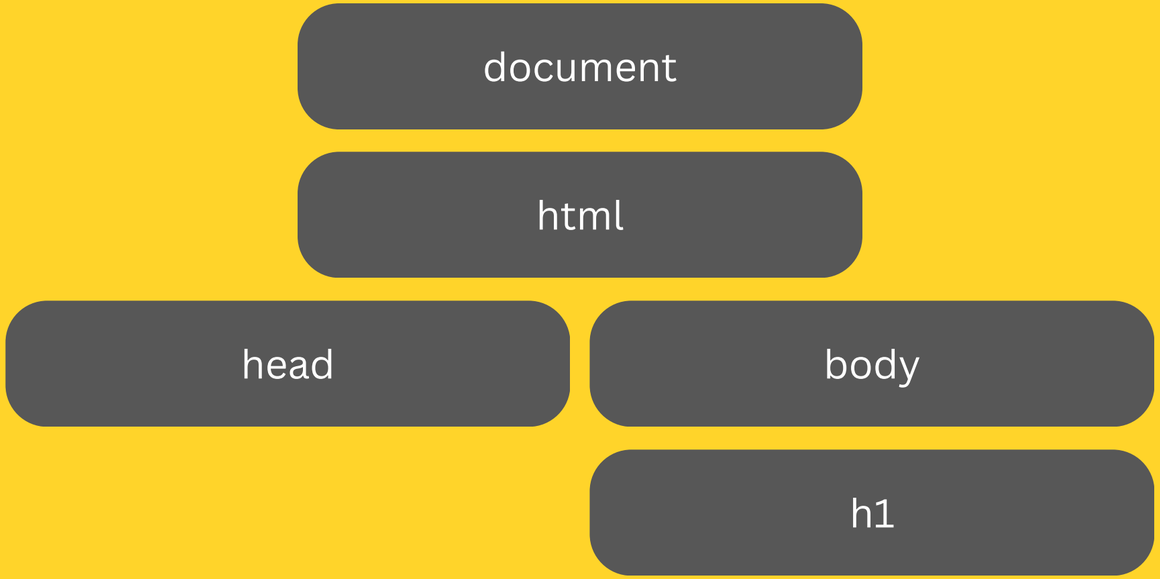
console.log(task1.getInfo());
console.log(task2.getInfo());

task1.complete();
task2.changePriority(1);

console.log(task1.getInfo());
console.log(task2.getInfo());
```

# model DOM

DOM to Document Object Model - obiektowa reprezentacja dokumentu HTML.



# model DOM

```
console.log(document);
console.log(document.documentElement);
console.log(document.body);
console.log(document.head);
```

# model DOM

```
<body>
    <header>
        <h1>Sklep komputerowy</h1>
    </header>

        <section id="products">
            <h2>Produkty</h2>
            <p class="product">Klawiatura</p>
            <p class="product">Mysz</p>
        </section>

        <footer>
            <p>Kontakt: kontakt@sklep.com</p>
        </footer>
</body>
```

# document.querySelector()

Zwraca pierwszy element z dokumentu, który odpowiada zadanemu selektorowi CSS np.

```
const section = document.querySelector(".products");
const products = document.querySelectorAll(".products");

console.log(section.parentElement);
console.log(section.children);
console.log(section.firstElementChild);
console.log(section.lastElementChild);
console.log(section.children.length);
```

# getElementBy...

```
document.getElementsByClassName("product");
document.getElementsById("products");
```

# getElementBy...

```
document.getElementsByClassName("product");
document.getElementsById("products");

document.getElementsById("products").textContent;
```

# zadanie!

W dokumencie zadanie-dom.html:

- Znajdź nagłówek strony za pomocą getElementById()
- Znajdź sekcję z zadaniami za pomocą querySelector()
- Pobierz wszystkie zadania i wyświetl ich liczbę
- Wyświetl tekst każdego zadania w osobnej linii konsoli
- Policz zadania oznaczone klasą important
- Sprawdź, czy istnieje element o id='task-form'
- Wyświetl tekst pierwszego i ostatniego zadania

# zadanie!

Utwórz stronę internetową przedstawiającą katalog trzech książek. Każda książka powinna być elementem *article* z klasą *book* i zawierać tytuł, autora oraz numer ISBN. Następnie napisz skrypt, który:

- Wyszuka wszystkie książki w dokumencie.
- Wyświetli liczbę znalezionych książek.
- Wyszuka pierwszą książkę i wyświetli jej tytuł.
- Wyszuka książki oznaczone klasą recommended.
- Sprawdzi, czy istnieje element o identyfikatorze book-form, i wyświetli odpowiedni komunikat.

# modyfikowanie DOM

JavaScript pozwala nie tylko odczytywać elementy dokumentu HTML, ale również je zmieniać. Możemy modyfikować:

1. Zawartość tekstową i HTML.
2. Atrybuty elementów.
3. Style CSS.
4. Klasy przypisane do elementów.

# textContent vs innerHTML

```
const title = document.querySelector("h1");

title.textContent = "Nowy tytuł strony";
```

interpretowane jako tekst

```
const box = document.querySelector("#box");
box.innerHTML = "<h2>Tytuł akapitu</h2><p>Nowy akapit</p>";
```

interpretowane jako kod HTML

# atrybuty elementów

Atrybuty HTML określają dodatkowe właściwości elementów, np. src, href, alt, id.

```
const img = document.querySelector("img");

console.log(img.getAttribute("src"));

img.setAttribute("src", "kot.jpg");

img.removeAttribute("alt");
```

odczyt

ustawianie

usuwanie

# atrybuty elementów

Do wielu atrybutów można uzyskać dostęp bezpośrednio, np.

```
const img = document.querySelector("img");
img.src = "kot.jpg";
```

# pola formularza

Właściwość value pozwala odczytywać i ustawiać wartości pól formularza.

```
<input id="username" value="Adam">
<script>
    const input = document.querySelector("#username");
    input.value = "Kasia";
</script>
```

# zadanie!

Utwórz stronę HTML o swoim ulubionym gatunku nosorożca, zawierającą:

- Nagłówek h1 z polska nazwą gatunku
- 2 akapity z krórkim opisem
- Zdjęcie
- Link do artykułu na wikipedii
- Pole tekstowe z adresem e-mail “Zapisz się na newsletter o nosorożcach”

# zadanie!

Napisz skrypt javascript, który:

1. Doda do nagłowka nazwę łacińską
2. Zmieni opis, pogrubiając wybrane słowa za pomocą innerHTML
3. Podmieni zdjęcie
4. Zmodyfikuje link tak, żeby otwierał sie w nowej karcie
5. Odczyta adres e-mail i wyświetli go w konsoli
6. Zablokuje możliwość edycji pola e-mail

# modyfikacja stylów

Właściwość style pozwala modyfikować style CSS elementu bezpośrednio z JavaScriptu.

```
const box = document.querySelector("#box");

box.style.backgroundColor = "red";
box.style.color = "white";
box.style.fontSize = "20px";
box.style.padding = "15px";
```

# classList

Właściwość classList pozwala zarządzać klasami CSS elementu.

```
const box = document.querySelector("#box");

box.classList.add("active");
box.classList.remove("hidden");
box.classList.toggle("highlight");
console.log(box.classList.contains("active"));
```

# zadanie!

Za pomocą skryptu javascript na stronie o nosorożcach:

1. Zmień kolor nagłówka na niebieski.
2. Ustaw wielkość czcionki akapitów na 18px.
3. Dodaj klasę highlight do pierwszego akapitu.
4. Ustaw szerokość obrazka na 300px.
5. Sprawdź, czy nagłówek ma klasę important. Jeśli nie, dodaj ją.

# tworzenie elementów

Metoda document.createElement() pozwala utworzyć nowy element HTML za pomocą JavaScriptu. Utworzony element nie jest jeszcze widoczny na stronie, musimy dodać go do dokumentu.

```
const paragraph = document.createElement("p");

paragraph.textContent = "Nowy akapit";
paragraph.classList.add("description");
```

# append() i prepend()

Metody append() i prepend() pozwalają dodawać elementy do dokumentu:

- append() — dodaje na koniec kontenera.
- prepend() — dodaje na początek kontenera.

```
const list = document.querySelector("#list");

const li = document.createElement("li");
li.textContent = "Nowy produkt";

list.append(li);
```

# before() i after()

Metody before() i after() pozwalają wstawiać elementy obok istniejącego elementu.

- before() — wstawia przed elementem.
- after() — wstawia za elementem.

```
const title = document.querySelector("h1");

const paragraph = document.createElement("p");
paragraph.textContent = "Nowy tekst";

title.after(paragraph);
```

# przykładowa struktura

```
const article = document.createElement("article");
const title = document.createElement("h2");
const description = document.createElement("p");

title.textContent = "Nowy produkt";
description.textContent = "Cena: 100 zł";

article.append(title, description);

document.body.append(article);
```

dodanie zawartości artykułu - tytułu i opisu

ważne: dodanie elementu do body

# zadanie!

Utwórz stronę przedstawiającą katalog książek. Dana jest tablica:

```
const books = [
    { title: "Hobbit", author: "J.R.R. Tolkien", year: 1937 },
    { title: "Diuna", author: "Frank Herbert", year: 1965 },
    { title: "Solaris", author: "Stanisław Lem", year: 1961 }
];
```

Za pomocą skryptu javascript:

1. Dla każdej książki utwórz element article.
2. Dodaj nagłówek h2 zawierający tytuł.
3. Dodaj akapit z autorem i rokiem wydania.
4. Nadaj każdemu artykułowi klasę book.
5. Umieść wszystkie artykuły w kontenerze (div) #books.

# remove()

Metoda remove() usuwa element z dokumentu.

```
const paragraph = document.querySelector("p");
paragraph.remove();
```

# replaceWith()

Metoda replaceWith() pozwala zastąpić istniejący element innym.

```
const oldTitle = document.querySelector("h1");

const newTitle = document.createElement("h2");
newTitle.textContent = "Nowy nagłówek";

oldTitle.replaceWith(newTitle);
```

# replaceChildren()

Metoda replaceChildren() usuwa dotychczasową zawartość elementu i zastępuje ją nową.

```
const list = document.querySelector("#list");

const li = document.createElement("li");
li.textContent = "Jedyny element";

list.replaceChildren(li);
```

podmiana zawartości listy na tylko jeden element

Aby usunąć wszystkie elementy potomne:

```
list.replaceChildren();
```

# zadanie!

Za pomocą skryptu javascript na stronie katalogu książek:

1. Usuń pierwszą książkę
2. Zastąp drugą książkę nową
3. Przenieś ostatnią książkę na początek katalogu
4. Wyświetl w konsoli liczbę pozostałych książek

# model zdarzeń w javascript

Zdarzenia to:

- sygnały wysyłane przez przeglądarkę informujące o tym, że coś się wydarzyło
- główny sposób w jaki skrypt może reagować na działania użytkownika

Przykłady zdarzeń: kliknięcie myszką (click), wysłanie formularza (submit), wciśnięcie klawisza na klawiaturze, najechanie na element

# wybrane zdarzenia

- click - pojedyncze kliknięcie lewym przyciskiem myszy
- dblclick – podwójne kliknięcie
- contextmenu – kliknięcie prawym przyciskiem
- mouseenter – najechanie kursorem w obszar elementu
- mouseleave – opuszczenie obszaru elementu przez kursor
- keydown – moment wciśnięcia dowolnego klawisza
- keyup – moment zwolnienia wciśniętego klawisza
- submit – próba wysłania formularza
- focus – aktywacja pola formularza
- blur – utrata ostrości (np. kliknięcie poza aktywne wcześniej pole)
- input – wywoływane przy każdej, nawet najmniejszej modyfikacji wartości (np. wpisanie pojedynczej litery)
- load – moment w pełni zakończonego ładowania strony (włącznie ze wszystkimi zdjęciami i arkuszami CSS)
- resize – zmiana rozmiaru okna przeglądarki.
- scroll – przewijanie widoku dokumentu lub konkretnego kontenera.
- copy – skopiowanie treści

# nasłuchiwanie zdarzeń

```
const btn = document.querySelector("#myBtn");
```

zdarzenie którego nasłuchujemy - click

funkcja wykonywana po wystąpieniu zdarzenia

```
btn.addEventListener("click", () => { console.log("Przycisk został kliknięty!"); });
```

# obiekt event (e)

Każda funkcja wywoływana przez zdarzenie otrzymuje specjalny obiekt klasy Event.

event.**target** - właściwość wskazująca na dokładny element na stronie, który wywołał zdarzenie (np. konkretny kliknięty akapit)

event.**type** - nazwa zdarzenia, np. "click" lub "submit"

# event.preventDefault();

Niektóre elementy HTML mają wbudowane, domyślne zachowania (np. kliknięcie w przycisk typu submit wewnątrz formularza przeładowuje cały dokument), które możemy zablokowac metodą event.preventDefault() - blokuje domyślną akcję, pozwalając nam np. walidować dane bez odświeżania strony.

```
form.addEventListener("submit", (e) => {
    e.preventDefault();
});
```

# zadanie!

Zadanie w pliku zadanie-darkmode.txt

# formularze i kontrolki w javascript

Dostep do formularza możemy uzyskać poprzez:

- getElementById lub querySelector np. *document.querySelector('#form-id')*

- document.forms['nazwa-formularza'] np.

```
const formularz = document.forms['zamowienie'];
const loginInput = formularz.elements['username'];
```

po pobraniu formularza możemy zyskać dostęp do inputów poprzez atrybut elements

# pola tekstowe

Właściwość **value** przechowuje aktualnie wpisany tekst w polach text, password, email, number oraz textarea. Zawsze jest to typ danych string, więc przy wartościach liczbowych konieczne jest castowanie na typ numeryczny.

```
const wpisanyTekst = formularz.elements['miasto'].value;
const wiek = Number(formularz.elements['wiek'].value);

formularz.elements['miasto'].value = "Szczecin";
```

# pola wyboru

Do odczytania stanu pól typu **checkbox** lub **radio** służy właściwość **checked** (true lub false). Dla grup radio można użyć selektora sprawdzającego zaznaczenie:

```
const czyFaktura = formularz.elements['faktura'].checked;

const wybranaDostawa = formularz.querySelector('input[name="dostawa"]:checked')?.value;
```

w tym przypadku do zmiennej zostanie przypisana wartość zaznaczonego pola

znak zapytania zabezpiecza przed błędem czytania wartości z null

# pola wyboru

Dla grup radio można użyć selektora sprawdzającego zaznaczenie:

```
<form id="mojFormularz">
    <label>
        <input type="radio" name="dostawa" value="kurier">
        Kurier
    </label>
    <label>
        <input type="radio" name="dostawa" value="paczkomat" checked>
        Paczkomat
    </label>
</form>

    const wybranaDostawa =
    formularz.querySelector('input[name="dostawa"]:checked')?.value;
```

znak zapytania zabezpiecza przed błędem czytania wartości z null

w tym przypadku do zmiennej zostanie przypisana wartość zaznaczonego pola

# select

Lista rozwijana <select> również oddaje wybraną opcję przez właściwość .value

const wybranyKraj = formularz.elements['kraj'].**value**;

# zadanie!

Na podstawie formularza z pliku zadanie-konfigurator.html:

1. Napisz skrypt, który podepnie się pod zdarzenie wysyłania formularza i zablokuje domyślne przeładowanie
2. Odczytaj wszystkie wybrane przez użytkownika wartości
3. Zbierz je w jeden obiekt JavaScript
4. Wygeneruj pod formularzem na stronie dynamiczne podsumowanie w postaci wypunktowanej listy <ul>

# walidacja formularza

- natychmiastowa informacja zwrotna dla użytkownika (lepszy UX)
- odciążenie serwera od przetwarzania pustych lub błędnie wypełnionych danych

# walidacja formularza

Przypomnienie - używanie atrybutów HTML: required, minlength, maxlength, min, max, pattern.

<input type="email" name="mail" required>
<input type="number" name="wiek" min="13" max="99">

# walidacja formularza w javascript

```
const haslo = formularz.elements['password'];
const errorSpan = document.getElementById('password-error');

if (haslo.value.length < 8) {
    errorSpan.textContent = "Hasło musi mieć minimum 8 znaków!";
} else {
    errorSpan.textContent = "";
}
```

# Constraint Validation API

- checkValidity() - metoda wywoływana na formularzu lub konkretnym polu. Sprawdza, czy element spełnia wszystkie reguły wpisane w atrybutach HTML (np. required, min). Zwraca bool.
- reportValidity() - jak wyżej, ale dodatkowo od razu fizycznie wyświetla natywny dymek z błędem w przeglądarce, jeśli pole jest niepoprawne.
- setCustomValidity("text") - pozwala podmienić domyślny komunikat przeglądarki na własny. Aby zdjąć błąd i pozwolić na wysłanie formularza, trzeba wywołać setCustomValidity("") z pustym stringiem.
- Właściwość .validity - zwraca obiekt z flagami logicznymi, które precyzyjnie określają rodzaj błędu z HTML (np. validity.valueMissing, validity.tooShort, validity.patternMismatch)

# Constraint Validation API

Przykład użycia:

```
if (poleKod.validity.tooShort) {
    poleKod.setCustomValidity("Kod jest za krótki, musi mieć
dokładnie 5 znaków.");
}
```

# funkcja wywoływana na zmianę w input

```
inputMail.addEventListener('change', () => {
    if (inputMail.validity.valid) {
        inputMail.classList.remove('klasa2');
        inputMail.classList.add('klasa1');
    } else {
        inputMail.classList.remove('klasa2');
        inputMail.classList.add('klasa1');} });
```

# testowanie regex

Metoda stringa .test() pozwala przetestować zgodność ze zdefiniowanym wyrażeniem regularnym.

```
const regexKod = /^[0-9]{2}-[0-9]{3}$/;
const wpisanyKod = "70-100";

if (regexKod.test(wpisanyKod)) {
    console.log("Kod jest poprawny!");
} else {
    console.log("Błędny format kodu!");
}
```

# zadanie!

Stwórz formularz z jednym polem na kod dostępu do warsztatu (wymagany, dokładnie 5 znaków). Wykorzystaj Constraint Validation API, aby zmienić domyślny komunikat przeglądarki na własny, używając metody setCustomValidity.